Linear Regression

Finding the Best-Fit Line from Data

Dr. Nikhilanand Arya

Indian Institute of Information Technology Allahabad

From Data Points to a Prediction Line

Can We Predict y From x?

Suppose we observe this...

$$(x_1, y_1),\ (x_2, y_2),\ \ldots,\ (x_m, y_m)$$

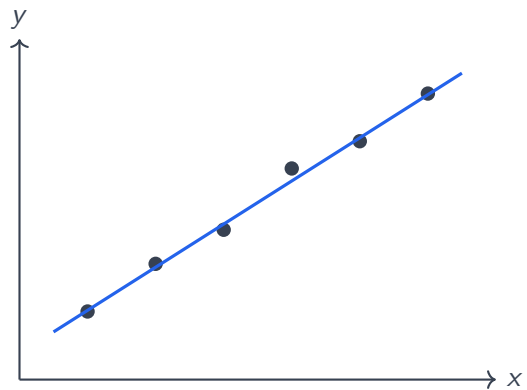
Our Question

Can we find a mathematical relationship between x and y that allows us to predict y for a new value of x?

Think

If the points roughly follow a straight-line pattern, what line should we draw?

Linear Regression: Find the Best-Fitting Line



Goal

Find the line that makes the predictions **as close as possible to the actual data**.

The Model: One Line, Two Parameters

$$\hat{y} = \theta_0 + \theta_1 x$$

θ_0 — Intercept

Value of $\hat{y}$ when

$$x = 0$$

θ_1 — Slope

How much $\hat{y}$ changes when x increases by 1.

Think

Changing θ_0 moves the line. Changing θ_1 rotates the line.

Prediction Is Easy... But Is It Good?

$$\hat{y} = \theta_0 + \theta_1 x$$

Suppose:

$$\theta_0 = 2, \qquad \theta_1 = 3$$

For $x = 4$:

$$\hat{y} = 2 + 3(4) = 14$$

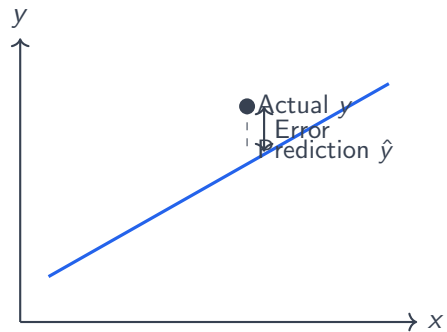
Prediction

The model predicts $\boxed{\hat{y} = 14}$.

But...

How do we know whether this prediction is good?

Actual vs Predicted: Where Is the Error?



$$e_i = y_i - \hat{y}_i$$

Key Question

How can we measure the error over the **entire dataset**?

How Bad Is Our Line?

Mean Squared Error

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} (\hat{y}_i - y_i)^2$$

- Compare prediction with actual value
- Square the error
- Add errors over all training examples
- Divide by the number of examples

Goal:

$$\min_{\theta_0, \theta_1} J(\theta_0, \theta_1)$$

Why Do We Square the Error?

$$e = y - \hat{y}$$

$$\boxed{e^2 = (y - \hat{y})^2}$$

Without Squaring

$$+5 + (-5) = 0$$

Errors could cancel each other.

With Squaring

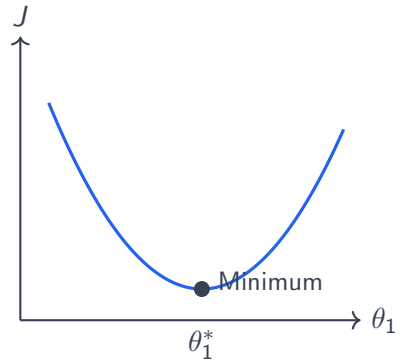
$$5^2 + (-5)^2 = 50$$

Large errors are penalized.

Intuition

Big mistakes should hurt more.

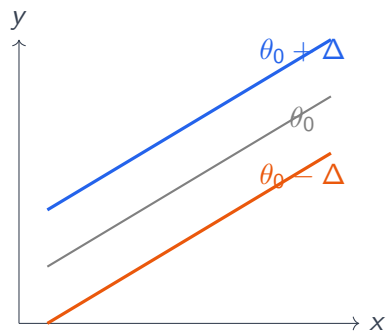
Change the Line → Change the Cost



Think

Which value of θ_1 gives the smallest error?

Parameter θ_0: Move the Line



Effect of θ_0

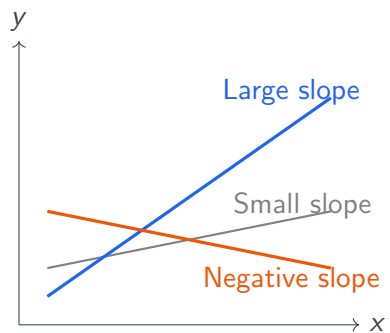
$$\hat{y} = \theta_0 + \theta_1 x$$

Changing θ_0 shifts the entire line vertically.

Remember

θ_0 controls the **intercept**.

Parameter θ_1: Rotate the Line



Effect of θ_1

$$\hat{y} = \theta_0 + \theta_1 x$$

Changing θ_1 changes the **slope** of the line.

Remember

θ_1 controls the **direction and steepness**.

How Do We Find θ_0 and θ_1?

We need to minimize the cost.

$$\boxed{J(\theta_0, \theta_1)}$$

Approach 1

Analytical Solution

Directly derive the optimal values of θ_0 and θ_1.

$$\theta_0^*, \theta_1^*$$

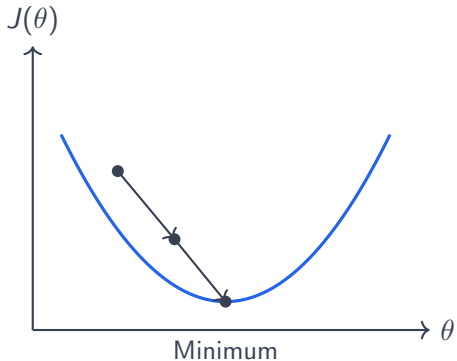
Approach 2

Gradient Descent

Start somewhere and iteratively move toward the minimum.

$$\theta \longrightarrow \theta^*$$

Gradient Descent: Walk Downhill



Intuition

Look at the slope

$\Downarrow$

Move downhill

Key Idea

The gradient tells us
which direction to move.

Gradient Descent Update

General Rule

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}$$

- $\theta_j \rightarrow$ parameter being updated
- $\alpha \rightarrow$ learning rate
- $\frac{\partial J}{\partial \theta_j} \rightarrow$ gradient

Key Idea

Gradient tells us the direction of increasing error.

Therefore, we move in the opposite direction.

For a Straight Line: What Are the Gradients?

$$\hat{y}_i = \theta_0 + \theta_1 x_i$$

Gradient for Intercept

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

Gradient for Slope

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i) x_i$$

Why Does x_i Appear in the Slope Gradient?

$$\frac{\partial J}{\partial \theta_1} = \frac{1}{m} \sum (\hat{y}_i - y_i) x_i$$

Think About One Example

$$\text{Contribution} = \text{Error} \times x_i$$

- Large $x_i \rightarrow$ stronger influence on slope
- Small $x_i \rightarrow$ weaker influence on slope
- Zero $x_i \rightarrow$ no contribution to slope gradient

Intuition

The slope determines how strongly x affects the prediction.

One Step of Learning

Predict → Measure Error → Compute Gradient → Update

Update

$$\theta_0 \leftarrow \theta_0 - \alpha \frac{\partial J}{\partial \theta_0}$$

$$\theta_1 \leftarrow \theta_1 - \alpha \frac{\partial J}{\partial \theta_1}$$

Repeat

Predict → Error → Gradient → Update

Can We Solve It Directly?

Instead of repeatedly updating the parameters...

$$\boxed{\theta_0 = \bar{y} - \theta_1 \bar{x}}$$

$$\theta_1 = \frac{\sum_{i=1}^{m} x_i y_i - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)\left(\sum_{i=1}^{m} y_i\right)}{\sum_{i=1}^{m} x_i^2 - \frac{1}{m}\left(\sum_{i=1}^{m} x_i\right)^2}$$

Numerical Challenge: Predict Sales

Year	2005	2006	2007	2008	2009
x	0	1	2	3	4
Sales y	12	19	29	37	45

Challenge

Find the linear regression equation and predict sales for the next year.

$$\boxed{\hat{y} = \theta_0 + \theta_1 x}$$

Numerical Solution

Given Sums

$$\sum x = 10 \qquad \sum y = 142$$

$$\sum xy = 368 \qquad \sum x^2 = 30$$

$$m = 5$$

Slope

$$\theta_1 = \frac{5(368) - 10(142)}{5(30) - 10^2}$$

$$\boxed{\theta_1 = 8.4}$$

Finding the Intercept

Use

$$\theta_0 = \bar{y} - \theta_1 \bar{x}$$

$$\bar{y} = \frac{142}{5} = 28.4$$

$$\bar{x} = \frac{10}{5} = 2$$

Therefore,

$$\theta_0 = 28.4 - (8.4)(2)$$

$$\boxed{\theta_0 = 11.6}$$

Now Predict the Next Year

Regression Model

$$\boxed{\hat{y} = 11.6 + 8.4x}$$

For the next year:

$$x = 5$$

Therefore,

$$\hat{y} = 11.6 + 8.4(5)$$

$$\boxed{\hat{y} = 53.6}$$

How Do We Find the Best-Fit Line?

$$\boxed{\hat{y} = \theta_0 + \theta_1 x}$$

For every training point (x_i, y_i):

$$e_i = y_i - \hat{y}_i = y_i - (\theta_0 + \theta_1 x_i)$$

We want the line with the smallest total squared error.

$$J(\theta_0, \theta_1) = \sum_{i=1}^{n} [y_i - \theta_0 - \theta_1 x_i]^2$$

Goal

Find θ_0 and θ_1 such that

$$\boxed{\frac{\partial J}{\partial \theta_0} = 0, \qquad \frac{\partial J}{\partial \theta_1} = 0}$$

Deriving θ_0 and θ_1

Step 1: Differentiate with respect to θ_0

$$J = \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i)^2$$

$$\frac{\partial J}{\partial \theta_0} = -2 \sum_{i=1}^{n} (y_i - \theta_0 - \theta_1 x_i) = 0$$

Therefore,

$$\sum y_i - n\theta_0 - \theta_1 \sum x_i = 0$$

$$\boxed{\theta_0 = \bar{y} - \theta_1 \bar{x}} \tag{1}$$

Deriving θ_0 and θ_1

Step 2: Differentiate with respect to θ_1

$$\frac{\partial J}{\partial \theta_1} = -2 \sum_{i=1}^{n} x_i (y_i - \theta_0 - \theta_1 x_i) = 0$$

$$\sum x_i y_i - \theta_0 \sum x_i - \theta_1 \sum x_i^2 = 0 \tag{2}$$

Deriving θ_0 and θ_1

Step 3: Substitute (1) into (2)

$$\sum x_i y_i - (\bar{y} - \theta_1 \bar{x}) \sum x_i - \theta_1 \sum x_i^2 = 0$$

Using

$$\sum x_i = n\bar{x}, \qquad \sum y_i = n\bar{y}$$

we obtain

$$\boxed{\theta_1 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n}(x_i - \bar{x})^2}}$$

$$\boxed{\theta_0 = \bar{y} - \theta_1 \bar{x}}$$

$$\boxed{\hat{y} = \underbrace{\bar{y} - \theta_1 \bar{x}}_{\text{Intercept}} + \underbrace{\theta_1}_{\text{Slope}} x}$$

What If One Feature Is Not Enough?

One x may not explain everything.

For example:

$$\text{House Price} = f(\text{Area}, \text{Bedrooms}, \text{Age}, \text{Location})$$

Multiple Linear Regression

$$\boxed{\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n}$$

Idea

Each feature gets its own weight.

What Does θ_j Mean?

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_3 x_3$$

θ_1	θ_2	θ_3
Effect of x_1	Effect of x_2	Effect of x_3

Think

The model learns how strongly each feature influences the prediction.

Gradient Descent: Multiple Features

$$\boxed{\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}}$$

For $j = 0$:

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i)$$

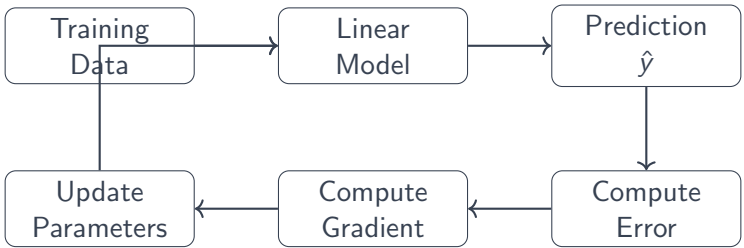
For $j \geq 1$:

$$\frac{\partial J}{\partial \theta_j} = \frac{1}{m} \sum_{i=1}^{m} (\hat{y}_i - y_i) x_{ij}$$

Important

All parameters are updated during each iteration.

The Complete Linear Regression Learning Cycle



The Learning Loop

Predict → Measure → Correct → Repeat

Linear Regression: Code

```
import numpy as np

class LinearRegression:

    def __init__(self, lr=0.01, n_iters=1000):
        self.lr = lr
        self.n_iters = n_iters
        self.weights = None
        self.bias = None
```

What are we storing?

Learning rate + number of iterations + model parameters

Training the Model

```
def fit(self, X, y):
    n_samples, n_features = X.shape
    self.weights = np.zeros(n_features)
    self.bias = 0
    for _ in range(self.n_iters):
        y_pred = np.dot(X, self.weights) + self.bias
        dw = (1/n_samples) * np.dot(X.T, y_pred-y)
        db = (1/n_samples) * np.sum(y_pred-y)
        self.weights -= self.lr * dw
        self.bias -= self.lr * db
```

Notice

The code is implementing exactly:

Prediction → Gradient → Update

Prediction

```
def predict(self, X):

    return np.dot(X, self.weights) + self.bias
```

$$\boxed{\hat{y} = Xw + b}$$

Simple Idea

Once the parameters are learned, prediction is just a weighted sum.

Can You Build the Best Line

x	1	2	3	4
y	3	5	7	8

Your Task

$$\theta_0 = 0, \qquad \theta_1 = 1$$

$$\alpha = 0.1$$

1. Calculate $\hat{y}$ for every point.
2. Calculate the errors.
3. Calculate $\frac{\partial J}{\partial \theta_0}$.
4. Calculate $\frac{\partial J}{\partial \theta_1}$.
5. Perform one parameter update.